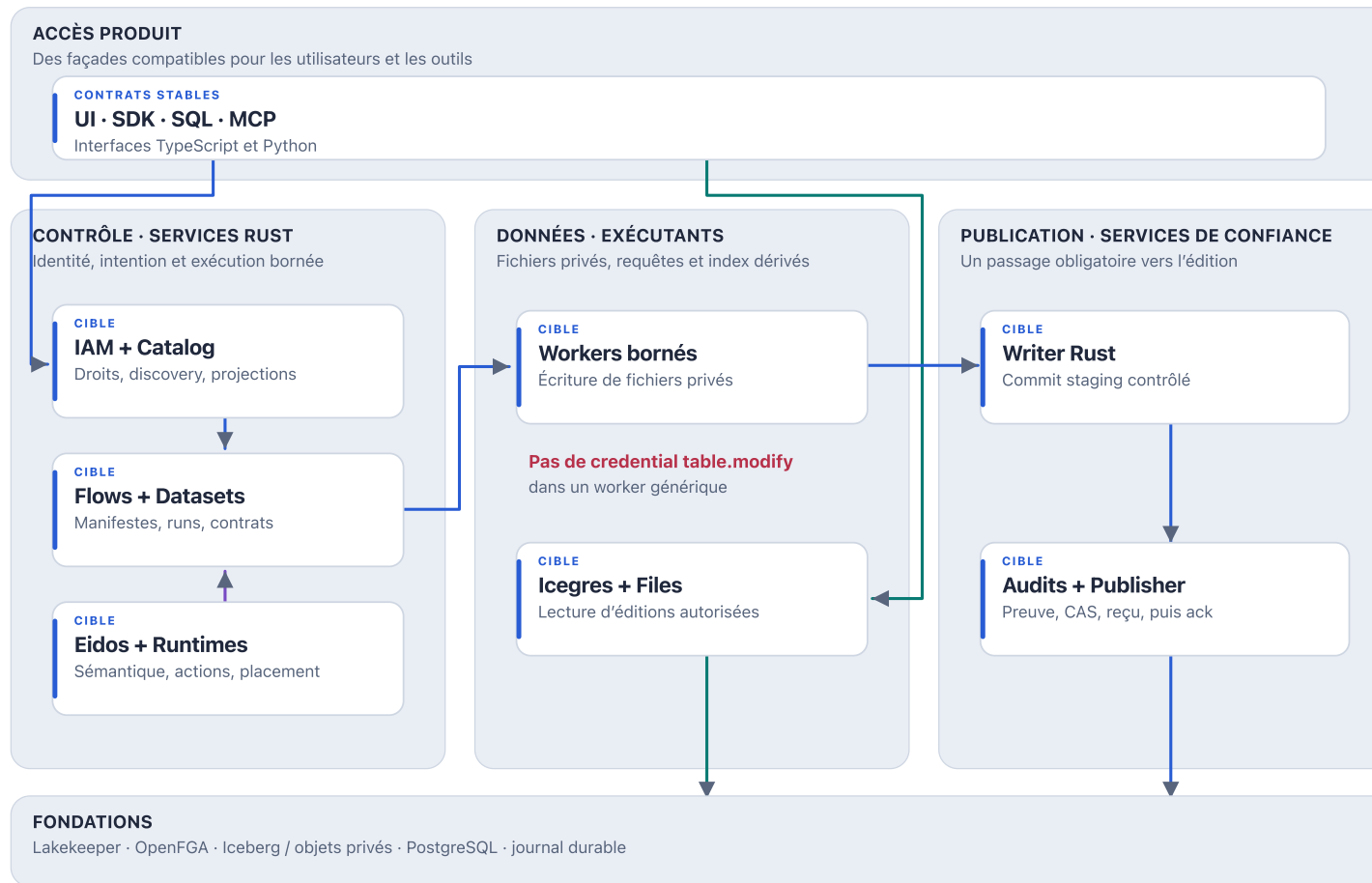


Une plateforme, une frontière de publication

Les services Rust organisent le travail. L'édition Iceberg publiée fixe ce que les utilisateurs peuvent lire et ce que les sources peuvent acquitter.



Les blocs bleus sont la cible. Les fondations existent ; leur assemblage et ses frontières restent à qualifier.

Une plateforme, une frontière de publication

Cible proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Flows organise ; l'édition fait foi

Cible proposée · Le scheduler possède l'intention et le suivi des runs, pas le curseur autoritaire.

PostgreSQL conserve commandes, leases et caches. La reprise relit origin.* depuis une édition publiée autorisée. Un worker prépare un résultat ; il ne peut ni publier main ni acquitter la source.

Conserver les usages

Cible proposée · Les interfaces existantes migrent par contrat et par cohorte.

UI, SDK, SQL, notebooks et MCP passent par des façades compatibles. Les chemins directs vers le catalogue ou S3 doivent respecter la même autorisation par édition. Aucun raccourci réservé à un client ne contourne le publisher.

Droits et projections

Cible proposée · IAM distribue les identités ; le Catalog produit des vues reconstruisibles des éditions.

OpenFGA évalue les relations. L'application de la politique se fait sur les accès effectifs aux éditions et à leurs objets. Les identités, clés et commandes ont leurs propres sauvegardes ; elles ne se reconstruisent pas toutes depuis Iceberg.

Eidos transforme le sens en plan

Cible proposée · Le modèle sémantique devient un plan épinglé que Flows peut exécuter.

Eidos compile types, concepts et actions. Runtimes choisit un exécutant qualifié. Le placement ne modifie ni l'autorité de publication, ni le checkpoint, ni les garanties requises.

Contrat de service

Manifest, Run, PreparedWrite, AuditEvidence, PublicationReceipt et InputPins constituent les coutures à stabiliser.

Frontière de confiance

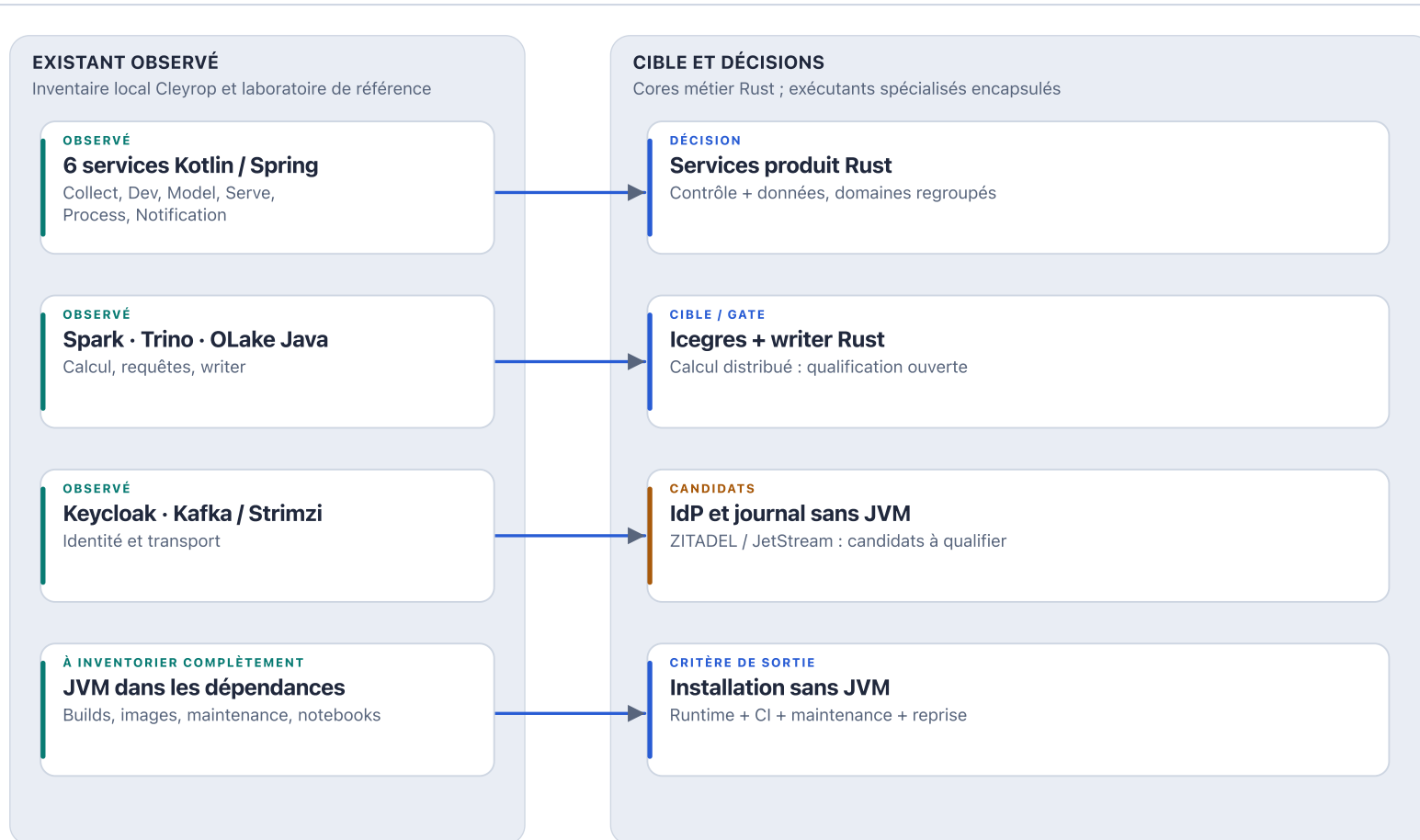
Le writer et le publisher ont des privilèges catalogue élevés. Leurs identités, accès réseau et chemins de commit doivent être fermés et testés.

Sources détaillées dans l'HTML : Étude consolidée · Droits Lakekeeper 0.12.0.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Architecture cible proposée · Chaque flèche représente un contrat, pas une intégration déjà livrée.

Retirer la JVM sans perdre les usages

La migration porte sur les services métier, les moteurs, l'identité et l'exploitation. Le changement de langage ne suffit pas à préserver les contrats.



Inventaire ≠ cible déployée · Les candidats IdP, journal et calcul distribué ne sont pas qualifiés dans cette étude.

Retirer la JVM sans perdre les usages

Inventaire + trajectoire proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Regrouper par contrat métier

Cible proposée · La cible redécoupe les responsabilités, sans imposer une traduction fichier par fichier.

Les six services Kotlin sont inventoriés. Les cores Python métier migrent aussi vers Rust selon le périmètre produit. Python/SQL des utilisateurs et interfaces TypeScript restent possibles. Les adaptateurs et exécutants Go/C++ sont encapsulés.

L'existant porte des contrats à conserver

Inventaire de code · Les API et objets métier sont le point de départ du découpage.

Data-collect porte la collecte. Data-dev porte manifestes, versions, graphes et intégration Git, avec les environnements de développement. Data-model porte l'identité, les métadonnées et les contrats des datasets ; il ne se confond pas avec la couche sémantique Eidos. Data-serve porte les accès, process-data les runs et notification-center les notifications. Un service existant peut alimenter plusieurs domaines cibles.

Des remplacements par workload

Inventaire et décision · Lire, transformer, écrire et maintenir Iceberg exigent des qualifications différentes.

Iceberg/DataFusion et DuckDB ne promettent pas toute la sémantique Spark/Trino. Ballista ou Sail restent des candidats de calcul distribué. OLake est réutilisé côté extraction Go ; son writer Java sort de la cible.

Identité et journal font partie de la migration

Inventaire de code · Le retrait JVM inclut les services tiers de la plateforme.

Migrer protocoles OIDC, droits, sessions et audit ; pour le journal, prouver la durabilité, le replay, la déduplication et la rétention avant de déplacer les acks. Aucune qualification complète des remplaçants n'a été exécutée ici.

Sélection des moteurs

Comparer seulement des configurations offrant les mêmes garanties de publication, de droits et de reprise.

Retrait vérifiable

Une installation neuve et une restauration sans JVM doivent fonctionner avant de clore le programme.

Sources détaillées dans l'HTML : Inventaire et migration · Code du laboratoire de référence.

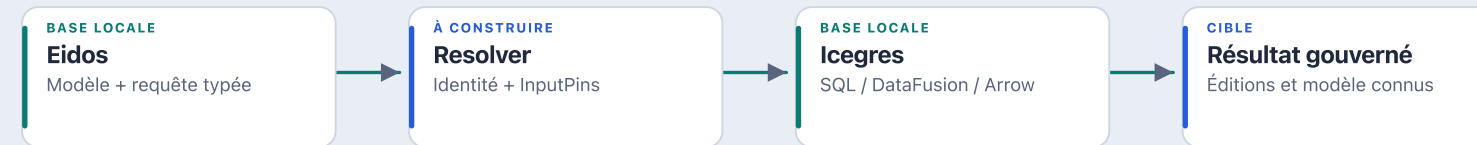
Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Inventaire ≠ cible déployée · Les candidats IdP, journal et calcul distribué ne sont pas qualifiés dans cette étude.

Icegres exécute. Eidos donne le sens.

Les deux projets apportent des briques Rust utiles. Leur intégration passe par des éditions épinglées et le même contrôle de publication que les autres services.

LECTURE · DU SENS AU RÉSULTAT

Une requête se fixe sur une version du modèle et des éditions de données



ACTION · DE L'INTENTION À L'ÉDITION

Une action peut créer un run ; elle ne devient pas un raccourci vers main



ADAPTATIONS À LIVRER

Les composants locaux sont observés ; ces coutures de gouvernance restent à implémenter

Eidos : authority + actions

Actif PostgreSQL / latest → pins d'édition

Icegres : query + write

Direct sync / buffer → PreparedWrite

Contrats communs

Politique · reçu · annulation

Le vert désigne une base locale observée. Le bleu désigne le contrat Hemera à construire.

Icegres exécute. Eidos donne le sens.

Réutilisation proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

InputPins avant exécution

Cible proposée · La réponse doit pouvoir expliquer quelles données et quelle définition l'ont produite.

Capturer l'édition du modèle sémantique, les UUID de tables, snapshot IDs ou release, la version du contrat et l'identité/politique. Les pins sont conservés pendant le run ; ni latest ni committed_at ne remplacent cette résolution.

Ce qu'Eidos apporte

Observé dans le code · Le compilateur sémantique et le typage évitent de reconstruire cette couche.

Le code local possède une sémantique et des plans d'action réutilisables. Les états actifs PostgreSQL et les recherches du dernier commit ne satisfont pas encore l'autorité d'édition Hemera. Les publications de modèle doivent devenir des éditions de définition gouvernées.

Ce qu'Icegres apporte

Observé dans le code · Le moteur Rust fournit une base de requêtes et d'écriture à intégrer.

Réutiliser les composants SQL/DataFusion, Arrow, accès catalogue et writer selon leurs capacités réelles. Les chemins locaux de synchronisation et d'écriture bufferisée ne démontrent pas WAP, le fencing ni un PublicationReceipt complet.

Un résultat explicable

Adaptation à livrer · La réponse expose les pins qui ont servi à la calculer.

Contrat Flight/HTTP à compléter : identité d'utilisateur, tenant, politique, pins, erreurs et annulation. Le service de lecture applique les droits de ligne/colonne avant sortie. Le protocole Arrow n'apporte pas lui-même cette gouvernance.

Premier lot conseillé

Un modèle Eidos épinglé lit une édition via Icegres ; une action produit un candidat privé puis une édition auditée.

Tests d'acceptation

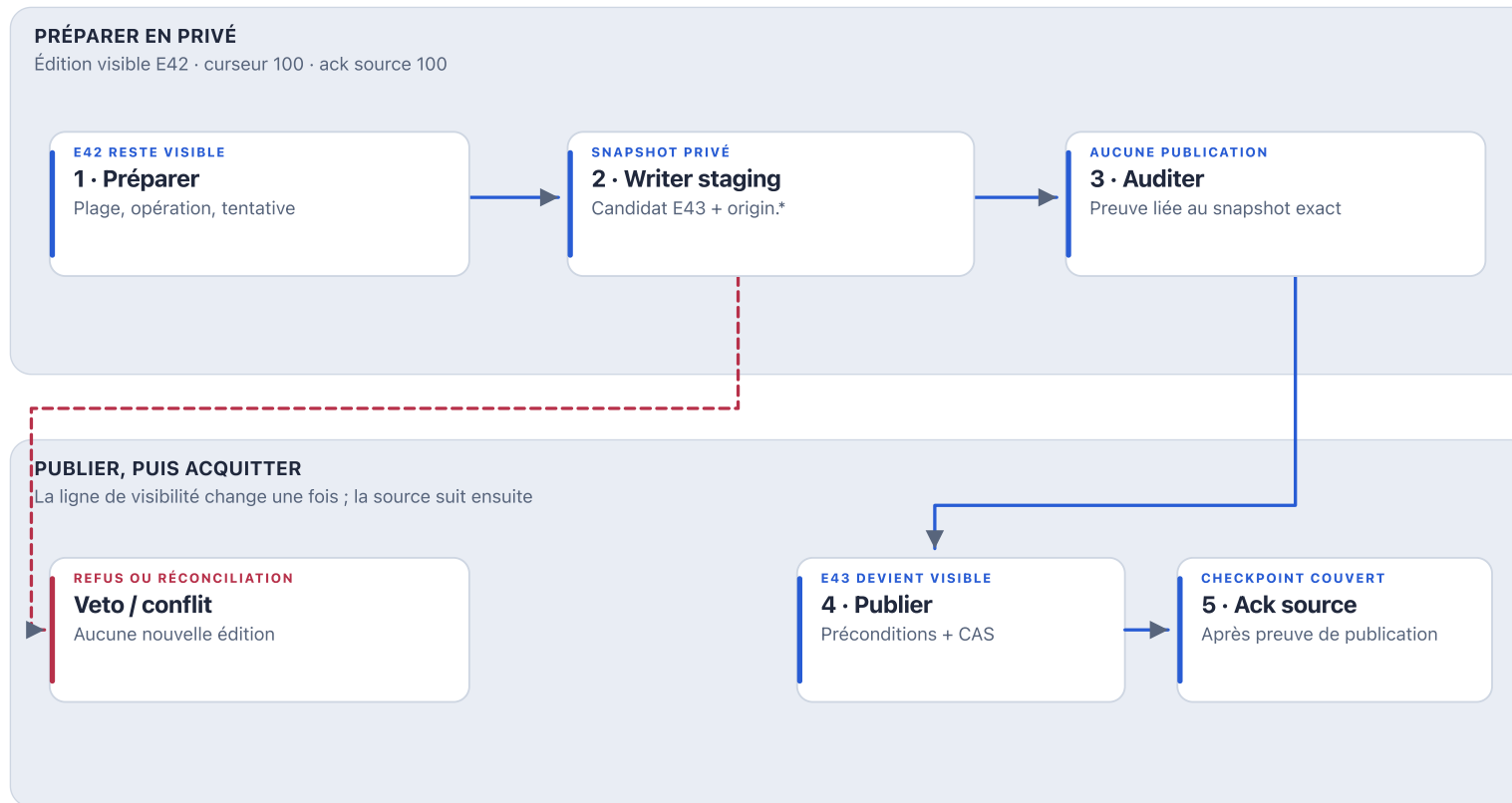
Concurrent publish, perte de réponse, annulation, tenant, politique de ligne/colonne, modèle périmé et lecture à édition fixe.

Sources détaillées dans l'HTML : Analyse des dépôts locaux · Écarts du prototype.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Lecture de code locale · Icegres et Eidos n'ont pas été modifiés ou déployés par cette étude.

Une édition avant chaque acquittement

Le moment décisif est la publication conditionnelle. Un veto, un conflit ou une réponse perdue ne doivent jamais être interprétés comme une autorisation d'acquitter.



RÉPONSE PERDUE ≠ COMMIT ÉCHOUÉ

Rechercher une publication approuvée ; un snapshot candidat ou un cache absent ne suffit pas.

Simulation pédagogique du contrat cible · Ces animations ne sont pas des résultats de tests.

Une édition avant chaque acquittement

Contrat cible · simulation · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Lire le dernier checkpoint publié

Contrat cible · Le run part de E42, qui couvre la source jusqu'à 100.

Identifier source, époque, plage, participants, operation_id, tentative et fence. Une lease SQL ne prouve pas que le commit d'un ancien worker sera refusé. Les sources éphémères doivent déjà disposer d'un journal durable.

Créer un candidat sans visibilité métier

Mécanisme à qualifier · E43 existe comme snapshot privé. Le consommateur continue de lire E42.

Le writer de confiance crée la branche et le snapshot avec origin*. Un worker sans table.modify ne peut pas changer main. La première création de table est un cas à qualifier séparément ; les preuves WAP positives actuelles partent d'un main déjà initialisé.

Lier la preuve au candidat

Contrat proposé à qualifier · L'audit porte sur le snapshot exact, ses entrées et son contrat.

Après création du candidat, produire une preuve immuable : UUID, snapshot ID, opération, contrat, résultats et hash. Le lien proposé utilise un objet préalloué immuable ou un manifeste de release ; ce mécanisme reste à implémenter et qualifier. Fast-forward ne réécrit pas le summary.

Contrôler ce qui a pu changer

À implémenter · La publication est conditionnelle à l'état attendu de la destination.

Contrôler UUID, main attendu, ref de tentative, ascendance, schéma/spec et droit de tentative. Les assertions REST disponibles ne constituent pas un assert-fence natif. Le refus d'un zombie au commit exige un mécanisme applicatif testé.

Écriture et audit

Le summary du candidat contient origin.* avant audit. Le support proposé pour la preuve post-audit et son lien immuable restent à implémenter et qualifier.

Récupération et histoire

Ne compter comme éditions que les publications approuvées. Un ancêtre de main peut être un snapshot intermédiaire jamais publié.

Sources détaillées dans l'HTML : Décision EL-4 · Contrat REST Iceberg 1.11.0.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Simulation pédagogique du contrat cible · Ces animations ne sont pas des résultats de tests.

WAP protège la visibilité. Il faut aussi protéger les octets.

Le banc a montré qu'un lecteur autorisé sur une table pouvait atteindre les fichiers d'un candidat non publié, même après retrait de sa branche. La cible doit contrôler l'accès à une édition et à tous ses objets.

CHEMIN REPRODUIT DANS LE BANC

Main reste intact ; la confidentialité staging échoue

OBSERVÉ

Lecteur : table.select

Autorisation au niveau de la table



OBSERVÉ

Métadonnées + signer

Le snapshot staging reste découvrable



ÉCHEC REPRODUIT

GET des objets non publiés

Accès aussi après retrait de la branche

CHEMIN CIBLE À QUALIFIER

Identité + édition + fermeture de fichiers autorisés

CIBLE

Identité et politique

Utilisateur, tenant, édition demandée



CIBLE

Service de lecture / PEP

Vérifie édition et fermeture d'objets



CIBLE

Accès objets limité

Données, manifests, deletes, statistiques

CONTRAINTES DU MODÈLE NATIF

Autre frontière : le droit de commit

table.modify permet aussi can_commit et can_drop. Un worker générique ne doit pas le recevoir.

Rouge : comportement reproduit dans le banc local · Bleu : frontière proposée, encore à qualifier.

WAP protège la visibilité. Il faut aussi protéger les octets.

Échec observé + correction proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

La confidentialité a échoué dans le banc

Échec reproduit · Main reste inchangé ; les octets du candidat sont lisibles avant et après retrait de sa branche.

STAGING-ISOLATION utilise une identité lectrice réelle, découvre le candidat via métadonnées puis lit par URL signée. Cette sonde ne déclenche pas d'audit : le veto est testé séparément dans WAP-VETO. Supprimer la ref ne suffit pas à révoquer les objets ou URLs déjà accessibles. Le résultat porte sur la configuration testée.

Un grant table ne désigne pas une édition

Observation du banc · Les droits relationnels sont utiles, mais leur granularité doit correspondre à la frontière produit.

Le test AUTH montre grants et révocation au catalogue. Il ne prouve pas l'isolation des snapshots d'une même table. Une restriction de ligne/colonne ne peut pas protéger un utilisateur qui possède en parallèle un accès objet plus large.

Les métadonnées donnent accès à d'autres chemins

Écart constaté · Masquer une branche dans l'interface ne ferme pas l'accès aux fichiers.

Contrôler métadonnées, manifests, fichiers de données, delete files, statistiques et anciennes versions accessibles. Le signer doit vérifier le graphe de fichiers de l'édition autorisée, pas seulement l'appartenance au stockage de la table.

Porter l'identité jusqu'à l'accès effectif

Cible non qualifiée · Le tenant et la politique suivent la requête jusqu'aux données retournées.

Pas de service account partagé transformant implicitement toute requête en lecture administrateur. Qualifier token exchange, expiration, sessions longues et révocation. Les conditions TTL du modèle natif testé sont refusées.

Acceptation confidentialité

Avec identité non privilégiée, aucune récupération des objets hors édition autorisée, avant et après veto, suppression de branche et expiration.

Acceptation publication

Un worker compromis ou périmé ne peut pas publier main, changer un schéma global ni supprimer une table via son accès staging.

Sources détaillées dans l'HTML : Résultats AUTH et STAGING-ISOLATION · Modèle natif Lakekeeper 0.12.0.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Rouge : comportement reproduit dans le banc local · Bleu : frontière proposée, encore à qualifier.

Huit types de datasets, une autorité d'édition

Tables, fichiers, modèles et index suivent le même principe de publication. L'export est un effet externe qui consomme une édition ; il n'ajoute pas un neuvième dataset.

DONNÉES TABULAIRES

Le snapshot publié porte les données et leur provenance

CIBLE

TABLE

Résultat tabulaire

CIBLE

INCREMENTAL

Évolution incrémentale

CIBLE

STREAM

Raw publié par lots bornés

DÉFINITIONS ET ARTEFACTS

Le snapshot peut publier un manifeste plutôt que les octets de l'artefact

CIBLE

VIEW

Définition versionnée

CIBLE

FILESET

Manifeste + objets privés

CIBLE

MODEL

Manifeste + artefacts

DÉRIVÉS POUR L'USAGE

Les index et stores sont des projections d'une édition autoritaire

CIBLE

EMBEDDINGS

Édition source → index

CIBLE

FEATURESET

Offline → store en ligne

SOURCE · OP · TEST

Nœuds de flow, pas des kinds

EFFETS EXTERNES

Une autre forme de reprise : journal d'effets, idempotence et réconciliation

EFFET

EXPORT

Édition consommée → livraison externe + reçu

Taxonomie cible : 8 kinds de datasets ; EXPORT est un effet, EXPOSURE déclare un consommateur. SOURCE, OP et TEST décrivent le flow.

Huit types de datasets, une autorité d'édition

Cible proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

STREAM conserve WAP

Contrat cible · Le raw forme des éditions bornées, auditable et rejouable.

La capture protocolaire peut être mutualisée et permanente ; les traitements restent des runs bornés. Les audits d'intégrité du raw précèdent sa publication et l'ack. Les audits métier aval s'appliquent à leurs propres éditions.

INCREMENTAL

Cible proposée · L'opération incrémentale publie un nouvel état complet de référence.

Un curseur n'est jamais validé par un simple succès de worker. Upsert, deletes et checkpoint vide doivent préserver les transactions de source et être testés avec le moteur cible.

Maintenance

La compaction peut publier une nouvelle édition sous WAP. Expiration et GC utilisent un plan audité puis une suppression protégée : une branche ne peut pas annuler un effacement physique.

TABLE

Cible proposée · Le résultat d'une transformation forme une édition Iceberg.

Le manifeste déclare entrées, schéma, contrat et audits. L'édition conserve les pins d'entrée et la provenance. Un veto n'autorise pas de bouton « forcer la publication ».

VIEW

Cible proposée · La définition, son contrat et ses références ont une édition autoritaire.

Le manifeste publié peut décrire une vue logique. Ne pas inventer un snapshot de données matérialisées lorsqu'il n'existe qu'une définition. Les entrées sont résolues selon un contrat explicite de pins.

Provenance

Une maintenance conserve le checkpoint source couvert et ajoute sa provenance d'opération. Elle ne se présente pas comme une nouvelle plage ingérée.

Sources détaillées dans l'HTML : Kinds, contrats et maintenance.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Taxonomie cible : 8 kinds de datasets ; EXPORT est un effet, EXPOSURE déclare un consommateur. SOURCE, OP et TEST décrivent le flow.

Migrer par preuves et par cohortes

Le chemin critique est la frontière d'édition. Les migrations d'identité, de journal et de calcul avancent en parallèle, puis se rejoignent sur une installation complète sans JVM.

DÉCISIONS DÉJÀ FIXÉES

Les choix ouverts portent sur les mécanismes et les candidats, pas sur ces exigences

DÉCIDÉ

Zéro JVM · cores Rust · éditions autoritaires

WAP · publish-then-ack · journal durable · exécutants bornés

CHEMIN CRITIQUE

Un gate se ferme par une preuve exécutée et conservée

GATE OUVERT

1 · Frontière d'édition

Writer / publisher / preuves
Confidentialité et reprise



APRÈS GATE 1

2 · Premier EL réel

Source → raw → ack
C1-C4 corrigés, EL-1/2/3/6



PAR CONTRAT

3 · Cohortes métier

Icgres, Eidos et services
API, données, droits, reprise



ACCEPTATION FINALE

4 · Sortie JVM

Installation + exploitation
Restauration et workloads

QUALIFICATIONS EN PARALLÈLE

Pas de calendrier annoncé avant mesure des écarts et des workloads

CHOIX OUVERT

Identité

OIDC, tenants, droits, TTL

CHOIX OUVERT

Journal

Durabilité, replay, ack

CHOIX OUVERT

Calcul distribué

SQL, mémoire, DML, erreurs

Dépendances de migration, sans calendrier estimé · Tous les gates de qualification cible ci-dessus restent ouverts.

Migrer par preuves et par cohortes

Trajectoire proposée · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Le lot qui conditionne les autres

Priorité 1 · Construire la publication contrôlée et prouver ses refus.

Cas requis : première table privée, worker sans accès main, candidat rejeté illisible, CAS concurrent, schéma concurrent, fence zombie, perte de réponse et reprise à cache vide. Un simple verrou local ne ferme pas le gate.

Ce qui ne reste pas à arbitrer

Décision confirmée · La cible finale exclut toute JVM et conserve les six invariants.

EL-4 est tranché : extraction OLake Go encapsulée, writer/publisher Rust. Si le writer direct n'est pas disponible, journal privé puis raw sous WAP. Pas d'exception STREAM publiant avant audit.

Un vertical complet avant généralisation

Priorité 2 · Prouver une source réelle jusqu'à l'ack, avec fautes injectées.

EL-1/2/3/6 d'abord, EL-8 avant dérive automatique. Qualifier transactions complètes, TRUNCATE, bootstrap/CTID, checkpoint vide et source partagée. Le prototype Python fournit des contre-exemples à fermer, pas le service Rust final.

Une cohorte, un propriétaire d'écriture

Trajectoire proposée · Basculer des usages complets avec un point d'autorité explicite.

Conserver façades compatibles ; shadow read possible. Un seul propriétaire de schedule, publication et ack. Séparer rollback logiciel, données et effets externes. Les changements de schéma suivent Expand / Migrate / Contract.

Reprise

Rétablir clés, identités, commandes et journaux en plus des éditions. Reconstituer le checkpoint depuis les publications autorisées, puis vérifier la couverture des acks.

Gouvernance de décision

Chaque gate possède scénario, environnement épinglé, preuve conservée et décision. Un candidat reste ouvert tant que ces éléments manquent.

Sources détaillées dans l'HTML : Backlog de reprise · Trajectoire détaillée · Gates et campagnes.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Dépendances de migration, sans calendrier estimé · Tous les gates de qualification cible ci-dessus restent ouverts.

Les preuves se lisent par contrat

Le banc apporte des résultats utiles et plusieurs refus nets. Il ne qualifie pas encore la plateforme cible : chaque contrat conserve ses limites et ses essais manquants.

33 entrées de registre, avec recouvrements

18 passed · 5 failed · 10 not_executed — aucun score de maturité

CONTRAT	OBSERVATION LIMITÉE	PREUVE QUI MANQUE
Publication WAP	CAS et staging après initialisation	Première table, zombie, crash / ack
Confidentialité	Candidat non publié lisible	Fermer l'accès aux objets non publiés
Expiration des droits	TTL natif Lakekeeper refusé	Intégrer modèle, contexte et caches
Parité SQL	Valeurs et types divergent	Contrat sémantique des moteurs finaux
Exécution et clients	Sous-ensemble Spark transitoire	Clients, tenants et moteurs sans JVM
Schéma et lectures	Ajout nullable / refresh observés	Contraction, concurrence et Icegres
Livraison et leases	Complétion SQL obsolète refusée	Fencing destination et replay réel
Rétention et GC	Propriété de rétention persistée	Expiration et purge physique sûres
Plan et compilation	Import, cache et contrat en défaut	Manifestes, fingerprints et wrappers

À PART : PROTOTYPE

Démo + 10 tests existants verts ; 4 contre-exemples reproduits, 0 garantie produit acceptée.

Familles de contrats, pas expériences indépendantes · Rouge : exigence en défaut dans le périmètre testé. Ambre : preuve partielle.

Les preuves se lisent par contrat

Audit des preuves conservées · 2 octobre 2026 · L'HTML associé contient tous les blocs, les scénarios et les références.

Publication conditionnelle et WAP

Partiel, avec contre-exemples · Les mécanismes ont été sondés ; la reprise complète reste ouverte.

I1/I2/I3/I4, WAP-VETO et INT-3 se recoupent. I4 écrit origin.state puis le relit sur le même handle : aucune perte de cache ni reprise de processus. Les WAP positifs partent d'une table initialisée. C1/C2 montrent première visibilité et promotion divergente du prototype.

Autorisation et confidentialité

Confidentialité refusée · Grants/révocations fonctionnent ; le staging n'est pas confidentiel dans ce banc.

AUTH et STAGING-ISOLATION sont exécutés. INT-2, INT-6 et INT-14 ne le sont pas. Les tests absents couvrent notamment identité utilisateur/tenant, token exchange et politiques SQL.

Expiration des droits

Intégration native refusée · Le test OpenFGA isolé et l'intégration native donnent des résultats différents.

OPENFGA-TTL accepte/refuse des contextes temporels fournis dans un store séparé. INT-5 constate que le modèle natif Lakekeeper rejette la condition. Aucun TTL intégré de bout en bout n'est validé.

Parité SQL

Parité refusée sur échantillon · Plusieurs cas de bord produisent des écarts.

I6 ne compare pas strictement les types. INT-1 utilise schémas/valeurs Arrow et normalise UTC. I8 est marqué passed sans assertion de parité : sa propre observation comporte une divergence substr à zéro. Ne pas le compter comme équivalence réussie.

Prochain banc

Identifiant commun de run, commits réellement mesurés, versions serveur détectées, répertoire de résultats isolé et journal d'exécution complet.

Limite de reproduction

Le banc précédent a été exécuté par étapes. Son script final assemblé n'a pas été rejoué intégralement. Cette relecture n'a pas relancé les campagnes réelles.

Sources détaillées dans l'HTML : Registre par famille · Résultats et limites · Preuves brutes.

Les mécanismes proposés ne sont pas des fonctionnalités déjà livrées. Familles de contrats, pas expériences indépendantes · Rouge : exigence en défaut dans le périmètre testé.

Ambre : preuve partielle.